ПРОЕКТНА ДОКУМЕНТАЦИЯ

Информационна система за управление на градски транспорт

База данни · SQLite · ER диаграма · Функционални изисквания

Проект: Transport Management Database

Версия на документа: 1.0 | Дата: 12.10.2026

Основа: предоставен SQL скрипт transport_database.sql и ER диаграма „Диаграма (1).pdf“.

Съдържание

1. Product Requirements

2. App Flow

3. Design Brief

4. Backend Schema

5. Бизнес правила и ограничения

6. Стратегия за начални данни

7. Индексация и производителност

8. Implementation Plan

9. Съответствие между диаграма и SQL

1. Product Requirements

Цел на проекта: проектиране на релационна база данни за управление на основните ресурси на градския транспорт: шофьори, превозни средства, маршрути, спирки и разписания. Проектът описва структурата и началните данни; графично приложение, потребителски роли и API не са реализирани в предоставения SQL.

1.1. Основни функционални изисквания

- Управление на шофьори: име, фамилия, категория на свидетелството и дата на назначаване.
- Управление на превозни средства: уникален регистрационен номер, тип, капацитет и година на производство.
- Управление на маршрути: уникален номер, начална и крайна точка.
- Управление на спирки: име и признак за крайна спирка.
- Свързване на маршрути със спирки и определяне на последователност на спирките.
- Създаване на разписания с конкретен маршрут, превозно средство, шофьор и час на тръгване/пристигане.

1.2. Обхват и потребители

Предвиден потребител е диспечер или администратор на транспортния оператор. В текущия проект е разработен само слой за съхранение на данни. Екраните, регистрацията, правата за достъп и билетната система са извън обхвата на наличния SQL.

1.3. Критерии за приемане

- Шестте таблици се създават успешно в SQLite.
- Предоставените INSERT заявки се изпълняват без грешка.
- Уникалните номера на маршрути и превозни средства не могат да се дублират.
- Невалидният капацитет и обратният ред на часовете се отхвърлят от CHECK ограничения.
- Невалидни външни ключове се отхвърлят при включено PRAGMA foreign_keys = ON.

2. App Flow

Тъй като няма предоставен интерфейс, следният поток е предложен бъдещ работен процес, а не вече реализирана функционалност.

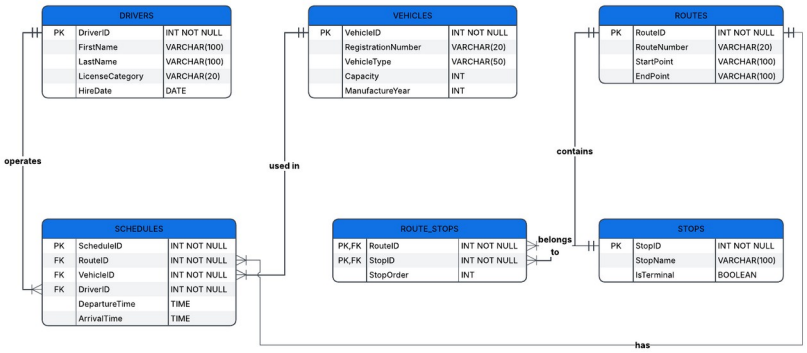
1. Администраторът добавя шофьори и превозни средства.
2. Създава маршрут с номер и начална/крайна точка.
3. Добавя спирки и ги свързва с маршрута посредством RouteStops и StopOrder.
4. Създава запис в Schedules, избирайки маршрут, превозно средство, шофьор и време.
5. Системата записва данните само ако ограниченията на базата са изпълнени.
6. Потребителят извлича разписания чрез SQL JOIN между съответните таблици.

3. Design Brief

Дизайнът на данните е релационен и модулен. Централният обект Schedules обединява три независими справочника — Drivers, Vehicles и Routes. Междинната таблица RouteStops реализира връзката много-към-много между маршрути и спирки.

За бъдещ потребителски интерфейс се препоръчват ясна навигация по модули, таблици с търсене и филтриране, форми за въвеждане и разбираеми съобщения за валидация. Това са предложения, които не присъстват в текущия проект.

3.1. ER диаграма



Фигура 1. ER диаграма, предоставена към проекта (1 страница).

4. Backend Schema

СУБД: SQLite. В скрипта се използва INTEGER PRIMARY KEY AUTOINCREMENT и PRAGMA foreign_keys = ON. Датите и часовете са представени с типовете DATE и DATETIME в декларациите, като SQLite работи с динамични типове и обичайно съхранява подадените стойности като текст в ISO формат.

Таблица	Първичен ключ	Назначение	Връзки
Drivers	DriverID	Информация за шофьори	1:N към Schedules
Vehicles	VehicleID	Автобуси и тролейбуси	1:N към Schedules
Routes	RouteID	Транспортни маршрути	1:N към Schedules, 1:N към RouteStops
Stops	StopID	Спирки	1:N към RouteStops
RouteStops	(RouteID, StopID)	Ред на спирките в маршрута	N:1 към Routes и Stops
Schedules	ScheduleID	Планирани курсове	N:1 към Routes,

4.1. Полета по таблици

Drivers

DriverID (PK), FirstName VARCHAR(50) NOT NULL, LastName VARCHAR(50) NOT NULL, LicenseCategory VARCHAR(10) NOT NULL, HireDate DATE DEFAULT CURRENT_DATE.

Vehicles

VehicleID (PK), RegistrationNumber VARCHAR(15) NOT NULL UNIQUE, VehicleType VARCHAR(30) NOT NULL, Capacity INTEGER NOT NULL, ManufactureYear INTEGER NOT NULL.

Routes

RouteID (PK), RouteNumber VARCHAR(10) NOT NULL UNIQUE, StartPoint VARCHAR(100) NOT NULL, EndPoint VARCHAR(100) NOT NULL.

Stops

StopID (PK), StopName VARCHAR(100) NOT NULL, IsTerminal INTEGER NOT NULL DEFAULT 0.

RouteStops

RouteID (PK, FK), StopID (PK, FK), StopOrder INTEGER NOT NULL.

Schedules

ScheduleID (PK), RouteID (FK), VehicleID (FK), DriverID (FK), DepartureTime DATETIME NOT NULL, ArrivalTime DATETIME NOT NULL.

4.2. Примерна справка

Примерен SQL за преглед на разписания с номера на маршрути и имената на шофьорите:

```
SELECT s.ScheduleID, r.RouteNumber, d.FirstName, d.LastName, v.RegistrationNumber,
s.DepartureTime, s.ArrivalTime
FROM Schedules AS s
JOIN Routes AS r ON r.RouteID = s.RouteID
JOIN Drivers AS d ON d.DriverID = s.DriverID
JOIN Vehicles AS v ON v.VehicleID = s.VehicleID;
```

5. Бизнес правила и ограничения

- Идентификаторите на основните обекти са цели числа с автоматично генериране (INTEGER PRIMARY KEY AUTOINCREMENT).

- RegistrationNumber и RouteNumber трябва да са уникални; техните полета са задължителни.
- Capacity трябва да е строго над 0 и най-много 200.
- ManufactureYear трябва да е 1990 или по-късна година.
- StopOrder трябва да е положително цяло число.
- Комбинацията (RouteID, StopID) в RouteStops е уникална; една спирка не може да се въведе два пъти в един и същ маршрут.
- Schedules изисква съществуващи RouteID, VehicleID и DriverID и DepartureTime < ArrivalTime.
- HireDate получава текущата дата, ако стойността е пропусната. IsTerminal получава 0 при пропусната стойност.

Важно: SQL не гарантира, че StopOrder е уникален в рамките на RouteID; не проверява дали категорията на шофьора е подходяща за типа превозно средство; не предотвратява припокриващи се курсове на един шофьор или автомобил. Това са допълнителни бизнес правила, които могат да бъдат реализирани впоследствие.

6. Стратегия за начални данни

Началните данни са демонстрационни и се добавят в ред, съобразен с външните ключове: първо Drivers, Vehicles, Routes и Stops; след това RouteStops и Schedules.

Таблица	Брой начални записи	Пример
Drivers	3	Ivan Dimitrov
Vehicles	3	A 1234 BB / Bus Solaris
Routes	3	B12 / Meden Rudnik → Slaveykov
Stops	5	Terminal Meden Rudnik
RouteStops	3	Маршрут 1, спирки 1→3→4
Schedules	3	25.10.2023 08:00–08:45

Общо: 20 реда. Демонстрационните разписания са с дата 25.10.2023 и не представляват актуално разписание. В скрипта липсва команда за изтриване на съществуващи таблици и повторното изпълнение върху вече запълнена база може да доведе до грешки.

7. Индексация и производителност

SQLite автоматично изгражда индекси за PRIMARY KEY и UNIQUE ограничения. Това включва уникалните полета RegistrationNumber и RouteNumber, както и комбинирания първичен ключ в RouteStops. В предоставения SQL няма допълнителни ръчно създадени индекси.

7.1. Препоръчителни допълнителни индекси (не са реализирани)

- CREATE INDEX idx_schedules_route_time ON Schedules(RouteID, DepartureTime);
- CREATE INDEX idx_schedules_driver_time ON Schedules(DriverID, DepartureTime);
- CREATE INDEX idx_schedules_vehicle_time ON Schedules(VehicleID, DepartureTime);
- CREATE INDEX idx_routestops_stop ON RouteStops(StopID);

Индексите са уместни при чести търсения по маршрут, шофьор, превозно средство и час на тръгване. За учебния набор от 20 реда осезаемо ускорение не се очаква. При по-голям обем ефектът следва да се проверява с EXPLAIN QUERY PLAN; всеки индекс добавя разходи при INSERT и UPDATE.

8. Implementation Plan

Етап	Дейности	Очакван резултат
1. Анализ	Уточняване на обектите, атрибутите и връзките	Списък с изисквания
2. Проектиране	Създаване на ER диаграма и релационен модел	ER схема
3. Имплементация	Създаване на шестте таблици и ограниченията	SQLite схема
4. Данни	Изпълнение на началните INSERT заявки	20 демонстрационни записа
5. Проверка	Тестове за FK, UNIQUE, CHECK и SELECT JOIN	Потвърдено функциониране
6. Развитие (по избор)	Интерфейс, отчети, допълнителни правила и индекси	Разширена информационна система

8.1. Инструкции за стартиране

7. Инсталирайте SQLite или отворете SQLite инструмент за работа с база данни.
8. Създайте нова база данни, например transport.db.
9. Отворете transport_database.sql и изпълнете скрипта в новата база.
10. Уверете се, че PRAGMA foreign_keys = ON е активно във всяка връзка, която трябва да проверява външни ключове.
11. Проверете резултата с SELECT COUNT(*) FROM Drivers; и аналогични заявки за останалите таблици.

9. Съответствие между диаграма и SQL

ER диаграмата и реалният SQL модел описват същите шест логически таблици и основните връзки, но има малки технически различия:

Елемент	ER диаграма	SQL скрипт
FirstName / LastName	VARCHAR(100)	VARCHAR(50)
LicenseCategory	VARCHAR(20)	VARCHAR(10)
RegistrationNumber	VARCHAR(20)	VARCHAR(15)
VehicleType	VARCHAR(50)	VARCHAR(30)
RouteNumber	VARCHAR(20)	VARCHAR(10)
IsTerminal	BOOLEAN	INTEGER DEFAULT 0
DepartureTime / ArrivalTime	TIME	DATETIME

Този документ третира SQL скрипта като източник на реално изпълняваната структура; ER диаграмата служи за концептуално представяне. Диаграмата е включена без редакции. При окончателно предаване е добре двете версии да се уеднаквят.

Заключение: проектът предоставя работещо ядро на база данни за градски транспорт, с ясно обособени обекти, релации и демонстрационни данни. Допълнителните функции за интерфейс, проверки за припокриване и оптимизация са бъдещи възможности, а не налични реализации.